



IN1030 cheatsheat - Oppsummering av pensum

Systemer, krav og konsekvenser (Universitetet i Oslo)



messages.pdf_cover_qr_code_label

IN1030 CHEATSHEET

Av Tuva Trøeng

DEL A

Rike bilder, brukerundersøkelser og datainnsamling, lover og regler og universell utforming.

RIKE BILDER

Hva er et rikt bilde?

Et rikt bilde handler om å representere ulike situasjoner/virkeligheter og få frem ulike interesser og interessenter. Det rike bilde inneholder også ulike relasjoner mellom interessentene og eventuelle konflikter. Formålet ved bruk av rike bilder å skape en forståelse av systemet og relasjonene mellom systemet og dens aktører.

Hva er en interessent?

En interessent er en person, gruppe eller organisasjon som kan påvirke eller bli påvirket av en sak eller en anliggende, f.eks. digital teknologi. En interessent er alle som har noe å si, eller har en interesse i systemet.

Eksempelvis kan en interessent i betalingssystemet Vipps være butikker som bruker Vipps som betalingsløsning. De tar i bruk systemet som en betalingsløsning, en interesse som blir møtt.

Hva er en interesse?

En interesse, eller concern, er faktiske eller forestilte behov som søkes tilfredsstilt av en interessent. Behov som man søker å bli møtt på.

Hva er en aktør?

I motsetning til interessent, er en aktør litt snevrere begrep. Mange interessenter kan også regnes som aktører.

En aktør er en betegnelse for en gruppe/personer/organisasjoner som faktisk anvender selve systemet. Et eksempel på dette er

Vi deler inn aktører i to forskjellige grupper:

1. Primæraktører har et eget mål i kommunikasjonen.
2. Sekundæraktører trengs for at primæraktøren skal nå målet og kommuniserer også aktivt med systemet.

Hva er forskjellen på en interessent og en aktør?

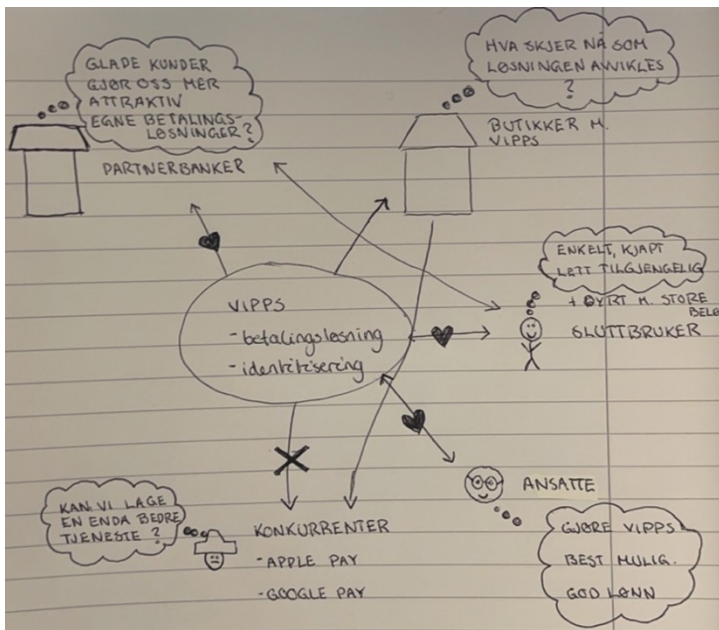
Det som skiller en aktør fra en interessent, er selve tilknytningen til systemet. En aktør regnes som en person/grupper/organisasjon som anvender systemet direkte. Dette i motsetning til en interessent som er en person/gruppe/organisasjon som blir påvirket eller påvirkes av systemet, både direkte og indirekte. Likevel er det viktig å nevne at en interessent, kan også være en aktør, hvis interessenten er en bruker av systemet.

Hvordan lage et rikt bilde?

Et rikt bilde skal inneholde interessenter, concerns/interesser, relasjoner og konflikter.

1. Interessenter. Tankebobler med interessene for systemet/situasjonen.
2. Hjerter mellom interessenter for relasjon.

3. Kryss for konflikter mellom interessenter.



Hvordan kan man bruke et rikt bilde i en systemutviklingsprosess?

Ved å bruke rike bilder i en systemutviklingsprosess vil man få en oversikt over situasjonen. Ut ifra å tegne et rikt bilde over en systemutviklingsprosess vil man få en oversikt over hvem som er involvert/interessenter og hvilke syn de har på situasjonen. Man får en overordnet oversikt over de ulike relasjonene, problemer/konflikter som kan oppstå mellom interessentene og ulike perspektiver på situasjonen. Derfor er det ikke dumt å tegne det rike bilde så tidlig som mulig i prosessen, slik at man kan starte med en god oversikt over prosjektet ved start.

User-Story(brukerhistorie)

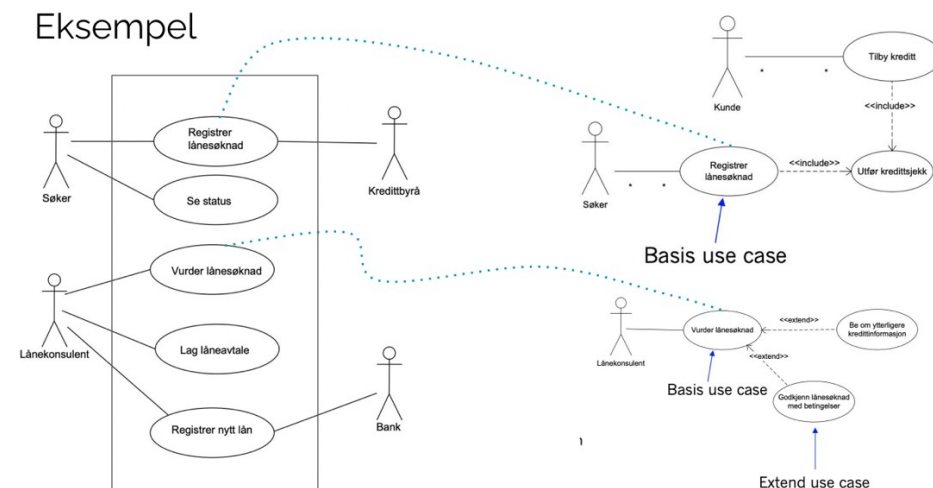
En User-Story er en historie fra brukerens perspektiv.

«Som bruker ønsker jeg å kunne betale billetten min ved faktura som betalingsløsning».

Use-Case(brukstilfelle)

Et Use-Case diagram viser interaksjon mellom et system og omgivelsene. Tar utgangspunkt i primæraktørs mål og hvordan sekundæraktører assisterer dette målet gjennom systemet.

Eksempeldiagram fra forelesning 29/3 2022



BRUKERUNDERSØKELSR OG DATAINNSAMLING

Hva er en brukerundersøkelse/undersøkelse av bruk og hvorfor gidder vi?

En brukerundersøkelse er.

Årsaken til at vi gjennomfører slike undersøkelser er mange. For eksempel ønsker man å utvikle nye produkter eller tjenester, forbedre noe som allerede eksisterer, eller øke forståelsen for noe, hvordan det fungerer i praksis.

Generelt ønsker man å tilegne seg mer og bedre kunnskap om et produkt eller en tjeneste.

Hvordan hadde du gått frem for å gjøre datainnsamling?

En datainnsamling er rett og slett innhenting av nyttig informasjon angående et produkt eller en tjeneste. Formålet med en slik undersøkelse er å innhente rådata til informasjon, slik at man får bedre forståelse for systemet/produktet/tjenesten. En slik innsamling består av flere faser.

Den første fasen handler om planleggingen av innsamlingen, og består av blant annet en pilotundersøkelse og et samtykkeskjema. Pilotundersøkelsen er en salgs testundersøkelse som skal avdekke feil og mangler ved metodebruk før man begynner med selve hoved datainnsamlingen. Ved å utføre en slik undersøkelse kan man optimalisere selve datainnsamlingen slik at man får best utbytte. Hvis man for eksempel skal intervju noen brukere av et system, kan man optimalisere for eksempel spørsmål som blir stilt.

Et samtykkeskjema er også nødvendig ved en slik datainnsamling. Det er en kontrakt mellom deltaker og systemutvikler over hvilke rettigheter deltakeren har i forhold til data som blir innsamlet.

Den andre fasen av prosessen, er innsamlingen av rådata. Dette kan skje ved observasjon, intervju eller et spørreskjema. Ved observasjon er det viktig å være så objektiv som mulig, og det finnes tre hovedtyper: skjult observasjon, åpen og passiv observasjon og deltagende observasjon.

Hvorfor er det viktig med pilotundersøkelser i en datainnsamlings-prosess?

Pilotundersøkelser er svært viktig i en datainnsamlingsprosess. Årsaken til dette er at en pilotundersøkelse vil være med på å avdekke eventuelle feil ved metoden som blir tatt i bruk. På denne måten kan selve datainnsamlingen optimalisere, for eksempel ved at spørsmål blir optimalisert eller eventuelle forhåndsbetingelser.

Samtykkeerklæring/samtykkeskjema

I brukerundersøkelser er samtykkeerklæring en kontrakt mellom deltaker i undersøkelsen og systemutvikleren. Denne kontrakten viser hvilke rettigheter deltaker har i forhold til personopplysningsdata som blir samlet inn i undersøkelsen. Hvis en tjeneste skal kunne ta i bruk personopplysninger fra en brukerundersøkelse, trenger tjenesten samtykke for deltakelsen fra deltakeren. I dagens samfunn kan personopplysninger ansees som svært viktig informasjon. Eksempelvis kan sosiale medier slik som Facebook ta i bruk personopplysningene dine for å fremheve reklame tilpasset deg. Med tilpasset reklame øker sannsynligheten for at man ønsket å kjøpe et produkt, og på denne måten kan selskaper «tjene» på personopplysningene dine.

For å ta i bruk personopplysningene, må et samtykkeskjema innfri flere krav.

Samtykkeskjemaet må samsvare med Personopplysningsloven, nærmere bestemt GDPR som krever at samtykket er frivillig, spesifikt, informert og utvetydig. (se under for forklaring).

Hvilken lov regulerer hva som må være med i en samtykkeerklæring?

Loven som regulerer hva som skal være med i en samtykkeerklæring finner vi i Personopplysningsloven, mer spesifikt GDPR (EUs personvernforordning). Her kreves det at samtykket for bruk og innsamling av personopplysninger skal være frivillig, spesifikt, utvetydig og informert.

Frivillig - at et samtykke er frivillig handler om at det ikke foreligger press for å samtykke eller at det foreligger negative konsekvenser dersom man ikke samtykker. Valget om å samtykke må være fritt.

Spesifikt - Man må også samtykke til et spesifikt formål. Det må være helt klart hva vedkommende samtykker til. Formålet må være lagt frem klart og tydelig.

Informert - Samtykke ansees kun som gyldig dersom man vet hva man samtykker til. Virksomheten er pålagt å informere om alt den enkelte trenger å vite for å foreta et reelt og informert valg. Samtykkeforespørselen må i det minste innholdet, hvem behandlingsansvarlig er, formålet, hvilke personopplysninger som samles inn, retten til å trekke tilbake samtykke og risiko og tiltak ved overføring av personopplysninger utenfor EU/EØS-området. I tillegg til dette må språket være enkelt og forståelig slik at målgruppen/brukeren enkelt forstår informasjonen.

Utvetydig – handler om at det ikke skal foreliggende noe tvil om at den enkelte gjennom en handling har ment å samtykke til bruk og innsamling av personopplysninger. For å samtykke må altså vedkommende foreta en aktiv handling, eksempelvis å hake av en boks eller underskrive et skjema. For eksempel regnes ikke som forhånds-avhakede bokser som et gyldig samtykke.

Kan en samtykkeerklæring skje muntlig?

En samtykkeerklæring kan gis både skriftlig, muntlig, gjennom bevegelser eller på andre måter. Det eneste kravet er at samtykke må kunne dokumenteres.

Hva skal et samtykkeskjema inneholde?

Et samtykkeskjema skal samsvare med Personopplysningsloven, som sier at samtykket skal være frivillig, utvetydig, informert og spesifikt. Frivillig – det skal være en villet handling uten noen negative konsekvenser, utvetydig – skal ikke være noe tvil om at vedkommende samtykker og samtykke skal skje gjennom en aktiv handling, informert – virksomheten må opplyse om alt vedkommende samtykker til for at det skal regnes som et frivillig valg og spesifikt – forteller nøyaktig hva personopplysningene brukes til.

Vedkommende skal også være opplyst om at det foreligger en mulighet for tilbaketrekning av samtykke, uten negative konsekvenser.

I tillegg regnes samtykket kun som gyldig dersom det er dokumentert.

Sekvenstabell

En sekvenstabell skal bestå av bruker og maskin, og brukes under observasjon for å kartlegge interaksjonen mellom bruker og systemet. Handling usynlig for maskin, handling synlig for maskin og effekt synlig for bruker, design rasjonale.

Bruker		Maskin	
Hadlings ikke synlig for maskinen	Handling synlig for maskinen	Effekt synlig for bruker	Design Rasjonale
Deltaker blir bedt om å vippe til en kontakt.			
«Da må jeg først logge inn»	Face-ID tas i bruk.		Oppfatter riktig ansikt, logges inn.
		Tas til hjem-skjermen på Vipps.	
«Også send»	Send-ikonet blir trykket på.	Gir deltaker valg om hvem som skal sendes til.	
«Så velger jeg den første her»	En av «nylige» kontakter blir valgt.	Gir deltaker valg om hvor mye penger som skal sendes.	
«Jaa, 10 kroner»	10 kroner blir lagt inn.		
	Det blir trykket på neste.	Kommer til en ny side som oppsummerer hvor mye du skal sende.	
«Da trykker jeg på send»	Det blir trykket på send.	Face-ID kommer opp som bekreftelse.	Face-ID godkjennes og pengene er sendt.

PERSONOPPLYSNINGSLOVEN, ARBEIDSMILJØLOVEN, ÅNDSVERKLOVEN OG ETIKK

Hva er en personopplysning? Hvilke personopplysninger har vi?

En personopplysning er en opplysning eller vurderinger som kan knyttes til en enkeltperson. Eksempler på typiske personopplysninger er navn, telefonnummer, epost-adresse, fødselsnummer eller biometri slik som fingeravtrykk eller hodeform. Registreringsnummeret på en bil kan også være en personopplysning, gitt at det ikke er en firmabil. En annen personopplysning er atferdsmønstre, som hvilke butikker du handler i, hvilke serier du ser på eller hvor du beveger deg i løpet av en dag.

Vi skiller på sensitive- og ikke-sensitive personopplysninger. De sensitive personopplysningene refereres til særlige kategorier personopplysninger, og er personopplysninger som er «viktigere» enn andre. Dette er blant annet fødselsnummer, kortdetaljer, passnummer eller kredittvurdering.

Hva er personvern?

Personvern handler om retten til et privatliv og retten til å bestemme over egne personopplysninger. Det omhandler også retten til å ha innflytelse på bruk og spredning av egne personopplysningene. Disse rettighetene, pliktene, lovene og reglene finner vi i Personopplysningsloven.

Personopplysningsloven + GDPR (personvernforordningen)

Personopplysningloven består av nasjonale regler og EUs personvernforordning, også kalt for GDPR – General Data Protection Regulation, som omhandler behandling – altså innsamling og bruk – av nettopp personopplysninger. GDPR ble implementert i loven i 2018, og er et sett med regler som gjelder for alle EU/EØS-land.

For å behandle personopplysninger er det et krav at vilkårene for behandling av personopplysninger er oppfylt. Et av de viktigste vilkårene er samtykke til behandlingen.

Dette gjøres ved at personen underskriver en samtykkeerklæring. I følge artikkel 4 i GDPR skal dette samtykke være frivillig, spesifikk, informert og utvetydig.

Hva mener GDPR med at samtykke kun er gyldig hvis det er enhver frivillig, spesifikk, informert og utvetydig viljesytring fra den registrerte?

Frivillig - at et samtykke er frivillig handler om at det ikke foreligger press for å samtykke eller at det foreligger negative konsekvenser dersom man ikke samtykker. Valget om å samtykke må være fritt.

Spesifikt - Man må også samtykke til et spesifikt formål. Det må være helt klart hva vedkommende samtykker til. Formålet må være lagt frem klart og tydelig.

Informert - Samtykke ansees kun som gyldig dersom man vet hva man samtykker til. Virksomheten er pålagt å informere om alt den enkelte trenger å vite for å foreta et reelt og informert valg. Samtykkeforespørselen må i det minste innholdet, hvem behandlingsansvarlig er, formålet, hvilke personopplysninger som samles inn, retten til å trekke tilbake samtykke og risiko og tiltak ved overføring av personopplysninger utenfor EU/EØS-området. I tillegg til dette må språket være enkelt og forståelig slik at målgruppen/brukeren enkelt forstår informasjonen.

Utvetydig – handler om at det ikke skal foreliggende noe tvil om at den enkelte gjennom en handling har ment å samtykke til bruk og innsamling av personopplysninger. For å samtykke må altså vedkommende foreta en aktiv handling, eksempelvis å hake av en boks eller

Hva menes med en databehandler og en behandlingsansvarlig i GDPR?

GDPR, eller personvernforordningen, skiller mellom behandlingsansvarlig og databehandler. Den behandlingsansvarlige har det overordnede ansvaret for å behandle personopplysningene i tråd med regelverket, og er dermed også den som har hovedansvaret for meldeplikt ved brudd på personopplysningssikkerheten.

En databehandler opptrer på vegne av den behandlingsansvarlige, det vil si at en databehandler kun kan behandle personopplysningene etter instruks fra den behandlingsansvarlige.

Er det sanksjoner for brudd på personopplysningsloven?

Brudd på personopplysningen er definert som et sikkerhets brudd som utilsiktet eller ulovlig tilintetgjøring, tap, endring, lagring, ulovlig spredning eller tilgang til personopplysninger. Dette bruddet bryter enten med konfidensialitet, integritet eller tilgjengelighet. Ved slike brudd har den behandlingsansvarlige hovedansvaret for meldeplikt til Datatilsynet.

Brudd på GDPR kan straffes med bøter på inntil 200 millioner kroner. For eksempel ble Østre Toten kommune ilagt en bot på 4 millioner kroner, for lekkasje av personopplysninger samt utilgjengelighet av personopplysninger pga. et hacker-angrep.

Hvilken rolle har Datatilsynet i GDPR?

Datatilsynet er et forvaltningsorgan som fungerer som både tilsyn og ombud. I forhold til GDPR – General Data Protection Regulations, så fører Datatilsynet kontroll ifht. at personopplysningsloven etterleves. De avdekker feil og mangler ved behandling av personopplysningene, noe som gjøres ved tilsyn og saksbehandling.

Arbeidsmiljøloven

Arbeidsmiljøloven regulerer hvordan arbeidslivet skal foregå, og hvilke rettigheter og plikter arbeidstakere og arbeidsgivere har. I denne loven nevnes også Personvern og som informatiker er spesielt paragraf 4.2 viktig som omhandler medbestemmelse. Paragrafen går

ut på at som en informatiker som jobber med å lage nye eller endre allerede eksisterende systemer i en bedrift, så skal brukere av systemene være med på å bestemme hvordan systemet skal være.

Åndsverkloven

Åndsverksloven regulerer rettighetene rundt åndsverk, det vil si kreative verk slik som musikk, kunst, teater, i tillegg til blant annet datamaskinprogram.

Datamaskinprogram regnes som et åndsverk, og har hovedregler om sier at «datamaskinprogram som er skapt av en arbeidstaker under utførelsen av oppgaver som omfattes av arbeidsforholdet» automatisk overføres til arbeidsgiver. Det vil si at arbeidsgiveren har opphavsretten på programmet, ikke selve arbeidstakeren.

Hva er etikk?

Etikk er læren om moral, hvordan man bør handle.

Etisk refleksivitet

Etisk refleksivitet handler om å kunne reflektere rundt sin rolle i for eksempel et utviklingsprosjekt. Det handler om å kunne se hvilken rolle man har fra et uteståendeperspektiv, og kunne diskutere rundt dette. Hvilket ansvar har man som systemutvikler? For eksempel, hvilket ansvar har man rundt universell utforming? Hvordan utvikle et system som inkluderer alle?

Etiske problemstillinger

Ved utvikling av programvare så er det viktig å tenke på hva som er rett og galt. Stille seg kritisk og være bevisst til utviklingen og bruken av teknologi.

UNIVERSELL UTFORMING OG LIKESTILLING- OG DISKRIMINERINGSLOVEN

Hva er universell utforming?

Universell utforming er et samfunns- og designmessig konsept som handler om at tjenester/produkter skal være tilgjengelig for alle i samfunnet. Samfunnet skal være utformet slik at så mange som mulig kan delta aktivt uavhengig av funksjonsevne. Når vi snakker om universell utforming er det lett å tenke på det fysiske aspektet, slik som at rullestolramper skal være tilgjengelig slik at rullestolbrukere lett skal kunne komme seg inn i et bygg. Men universell utforming er også et viktig prinsipp innen IT-tjenester. Det handler om at digitale tjenester også skal være tilgjengelig for allmennheten, uansett funksjonsnedsettelse. Dette er stadfestet i likestilling- og diskrimineringsloven, artikkel 18. Denne forskriften er begrenset til å gjelde nettløsninger, herunder digitale læremidler og automater og gjelder på alle samfunnsområder, med unntak av familieliv. Gjelder ikke i arbeidslivet.

Likestilling- og diskrimineringsloven

Likestilling- og diskrimineringsloven handler om at alle skal få delta med like vilkår på alle samfunnsarenaer, blant annet uavhengig av funksjonsnedsettelse. I forhold til IKT-løsninger er det paragraf 18 som er spesielt rett mot det. Der kommer det fram at både offentlige og private virksomheter har til plikt til universell utforming av IKT-løsninger, slik at alle kan bruke løsningen uavhengig av funksjonsnedsettelse.

Universell utforming og ulike tilpasninger

Hvis en webside er universelt utformet kan alle bruke siden på lik linje, uavhengig av funksjonsnedsettelse. Hvis nettsiden legger til rette for å tilpasse nettsiden etter behov blir den universelt utformet.

Slike tilpasninger kan for eksempel være **tilpasning av fargekontrast**. Det er viktig at det er en tydelig kontrast mellom bakgrunn og tekst, slik at også svaksynte og fargeblinde kan bruke denne løsningen på optimalt vis, på lik linje med alle andre.

En annen type tilpasning kan være at nettsiden kan brukes ved hjelp av **talefunksjon**. Hvis en bruker ikke kan navigere med hendene, kan brukeren navigere ved hjelp av taleevnen.

En tredje tilpasning kan være at systemet er mottakelig for ulike **hjelpemiddelteknologi**. Dette kan for eksempel være en stemmegenererende enhet.

WCAG – The Web Accessibility Guidelines

WCAG er en internasjonal anerkjent standard som brukes for universell utforming av IKT-løsninger, slik som apper og nettsider. Løsningene skal være tilgjengelig for alle uavhengig av ulike funksjonsnedsettelser. I Norge er det lovpålagt å følge forskrift om universell utforming av ikt-løsninger, offentlig sektor skal følge litt flere krav enn hva privat sektor gjør. Fra 1. februar 2023 ble det innført 12 nye krav som offentlig sektor skal følge.

WCAG 2.0 er bygd opp av fire overordnede prinsipper, i tillegg til at de understøttes av 12 retningslinjer og 61 testbare suksesskriterier.

1. **Mulig å oppfatte** – Dette prinsippet handler om at brukeren skal kunne oppfatte uansett hvilken måte brukeren oppfatter innhold, enten det er syn, lyd eller berøring. En retningslinje som understøtter dette prinsippet er retningslinje 1.4 som er mulig å skille fra hverandre. Det skal være enkelt for brukeren å se innholdet på siden, for eksempel er det viktig å kunne skille forgrunnen fra bakgrunnen. Dette kan implementeres ved bruk av farger som står i kontrast til hverandre, slik at tekst blir synligere og det blir enklere å lese for brukeren.
2. **Mulig å betjene** – Nettsiden skal betjene brukeren uansett hvordan brukeren navigerer, for eksempel ved bruk av tastaturet, en ekstern datamus eller ved hjelp av talefunksjon. En retningslinje som understøtter dette prinsippet er retningslinje 2.2 – nok tid. Mange nettsider baserer seg på tidsbaserte løsninger, ved for eksempel innlogging. Det er dermed viktig å tenke på at mange brukere trenger lengre tid enn andre. Dette kan implementeres ved en løsning som gir brukeren mulighet til å forlenge tiden, slik at brukeren får gjennomført handlingen.
3. **Forståelig** – Prinsipp nummer tre handler om at brukeren skal kunne forstå hvordan vedkommende skal navigere som fram på nettsiden, løse oppgavene og forstå informasjonen som blir gitt. Dette er understøttet av retningslinje 3.1 – leselig. Innholdet på siden skal være leselig og forståelig for brukeren. En mulig implementasjon av dette kan for eksempel være at brukeren har mulighet til å få hele betydningen av eventuelle forkortelse som er brukt på nettsiden.
4. **Robust** – Prinsipp nummer fire handler om at en nettside/digital løsning skal fungere sammen med ulike hjelpemidler, slik som forskjellige nettlesere og ikke minst ulike hjelpemiddelteknologi. Dette er understøttet av retningslinje 4.1 – kompatibel. Løsningen skal kunne samarbeide med ulike hjelpemiddelteknologi slik at ikt-løsningen er like tilgjengelig og funksjonell for brukere som er avhengig av slik teknologi.

Hvem er tilsyn for Universell Utforming?

UUtilsynet ved Digitaliseringsdirektoratet fører tilsyn etter WCAG-forskriften.

Brudd på universell utforming

UUTilsynet ved Digitaliseringsdirektoratet fører tilsyn, og sjekker om nettstedet/appen til en bestemt virksomheten følger kraene til universell utforming. Ved brudd på disse pålegger UUTilsynet krav om å rette opp i feilen, eller ved nødvendighet kan virksomheten bli ilagt dagmulkt, som er en «bot» virksomheten forplikter å betale for hver dag helt til feilen er rettet opp i.

Krav om Tilgjengelighetserklæring

I tillegg til kravet om at WCAG standarden skal følges, har det nylig kommet et krav om tilgjengelighetserklæring. En tilgjengelighetserklæring forteller om virksomheten lever opp til standarden om universell utforming, altså WCAG 2.1, og denne skal være godt synlig på deres nettside. Dette trådte i kraft fra og med 1. februar i år. Formålet med tilgjengelighetserklæringen er hovedsakelig at brukeren skal kunne være med på å påvirke den universelle utformingen av løsninger. Erklæringen gir en god oversikt over hvilke krav som er tilfredsstilt av løsningen, og er det noe brukeren ikke er fornøyd med kan vedkommende rapportere inn til virksomheten. I tillegg til dette vil denne erklæringen føre til en større bevissthet rundt universell utforming. For det første blir virksomhetene nødt til å sette seg inn i de ulike kravene i WCAG 2.1 for å kunne publisere tilgjengelighetserklæringen. I tillegg kan man tenke seg at dette blir som en slags motivasjon for å gjøre den universelle utformingen best mulig, da ingen ønsker er dårlig tilgjengelighetserklæring publisert for allmennheten.

Krav om Tilbakemeldingsfunksjon

I tillegg til kravene som framkommer av WCAG-standard, er det et krav at alle nettsider skal ha en tilbakemeldingsfunksjon. Tilgjengelighetserklæring har delvis som formål at brukeren skal kunne se hvilke tilpasninger som er gjort/mulige på nettsiden og kunne være med på å påvirke den videre utviklingen av den universelle utformingen. For at dette skal være mulig, må brukeren av nettsiden ha mulighet til på gi tilbakemeldinger, og derav kravet om en tilbakemeldingsfunksjon. Det skal være enkelt for brukeren å gi tilbakemelding.

Inkludering

Alle skal med. Man skal kjenne på en følelse av at man tilhører og er inkludert i et fellesskap. Kan sees på som essensen av universell utforming, alle skal med!

GAP-modellen

En modell som visualiserer hvordan en funksjonshemning er i samfunnet. Modellen peker på at funksjonshemninger er situasjonsbestemte, det er altså samfunnet som har et ansvar for å utvikle løsninger som gjør det enklere med samfunnsdeltakelse. Problemet ligger i gapet mellom samfunnets løsninger og personen med nedsatt funksjonsevne.

Nevn fire lover det er viktig å ta hensyn til som informatiker.

Som informatiker er det flere lover det er viktig å kjenne til for å gjøre en god jobb. Fire lover som er viktig å ta hensyn til som informatiker er **Personopplysningsloven**, **Likestillings- og diskrimineringsloven**, **Arbeidsmiljøloven** og **Åndsverksloven**.

Personopplysningsloven er viktig å kjenne til da den forteller hvordan personopplysninger skal behandles, via GDPR. Artikkel 4 i GDPR forklarer blant annet kravene rundt et samtykke, at det skal være frivillig, spesifikt, informert og utvetydig. Som informatiker er dette viktig å kjenne til da man ofte lager systemer som innhenter personopplysninger.

Likestillings- og diskrimineringsloven er også viktig å kjenne til i forhold til universell utforming. Paragraf 18 er spesielt rettet mot IKT-løsninger og sier at enhver IKT-løsning skal kunne brukes av alle på lik linje, uansett funksjonsnedsettelse.

Arbeidsmiljøloven er også en sentral lov, da paragraf 4.2 omhandler personvern, og at som bruker av et system har du medbestemmelsesrett over dine personopplysninger som blir samlet inn.

Åndsverkloven er også en sentral lov. Et åndsverk er et kreativt verk, noe også datasystemer regnes som. I denne loven kommer det fram at som ansatt i en virksomhet som har produsert et datasystem, er det arbeidsgiver som har opphavsretten på systemet og ikke den ansatte selv.

Hvordan kan informatikere jobbe med Universell Utforming? Gi ett eksempel.

Som informatiker er det svært viktig å kunnskap til Universell Utforming. Dette inkluderer blant annet WCAG-standarden. Årsaken til dette er for eksempel som utvikler må man ha kjennskap til de ulike lovene og reglene knyttet til universell utforming, for å kunne utvikle et system som alle kan bruke, uansett funksjonsnedsettelse. Har du ikke kunnskap til regelverket knytter til universell utforming, klarer du ikke utvikle et system som lever opp til standarden.

DEL B

Systemutviklingsprosessen, DevOps, kravhåndtering, informasjonssikkerhet og risiko, UML-modellering og IT-kontrakter.

SYSTEMUTVIKLINGSPROSESSEN

Hva er systemutvikling?

Systemutvikling handler om å utvikle, vedlikeholde og drifte datasystemer og informasjonssystemer. Alle teorier, metoder og verktøy for utvikling av programvare fra start til slutt.

Hva går systemutviklingsprosessen ut på og hvilke utfordringer kan man støte på?

Systemutviklingsprosessen handler om hvordan programvaren skal produseres, ved å ta i bruk ulike mekanismer for å styre, kontrollere og organisere arbeidet. Selve systemutviklingsprosessen er svært viktig for programvaren, da prosessen kan ha stor innvirkning på både produktivitet og kvalitet på programvaren. Eksempelvis passer noen prosessmodeller bedre til noen systemutviklingsprosesser enn andre.

I tillegg er det viktig med god systemutviklingsprosess fordi det påvirker, prosjektstyring, arbeidsmiljø, tidsbruk, evnen til å tilpasse seg endringer osv.

Hva er sammenhengen mellom systemutvikling og programmering?

Systemutviklingen kan ansees som hele arbeidsprosess rundt det å utvikle et nytt datasystem. Alt fra planleggingsprosessen, til selve programmeringen, til teammøter og dokumentering. Programmeringen er selve kildekoden bak programvaren, og kan sees som en del av systemutviklingen.

Prosessmodell

En prosessmodell er en modell som gir et overblikk over planlagt gjennomføring av et systemutviklingsprosjekt, fra start til slutt. Prosessmodeller kan deles inn to forskjellige kategorier: plandrevne prosessmodeller og smidige prosessmodeller.

Plandreven utvikling/plandrevne prosessmodeller (fossefallsmodellen)

Plandrevne prosessmodeller eller plandreven utvikling, kjennetegnes ved mye planlegging. Formålet er å planlegge prosessen så godt at man følger planen slavisk gjennom hele systemutviklingsprosessen. Utviklingsprosessen foregår steg for steg, hvor hvert steg

fullføres før man går videre til neste. Dette krever at man har jobbet godt med planleggingen på forhånd da det ikke er muligheter for å gå tilbake til det forrige steget. Eksempelvis når man har samlet inn alle kravene man trenger til systemet, og fortsetter til neste steg i utviklingen snur man ikke og endrer på kravene på nytt.

Smidig utvikling/smidige prosessmodeller

Smidige prosessmodeller eller smidig utvikling, kjennetegnes ved mer fleksibilitet enn plandreven utvikling. Her kreves det ikke en fullstendig plan på forhånd, inkludert absolutt alle krav. Man kan gjøre ferdig ett steg, revurdere selve systemet, og eventuelt endre på kravene og endre systemet deretter helt til man er fornøyd. Man tar høyde for å gå tilbake til forrige steg i prosessen og endre på kravene i et steg underveis.

Hva er hovedforskjellene mellom plandreven og smidig utvikling?

Hovedforskjellen mellom plandreven og smidig utvikling ligger i hvordan man organiserer systemutviklingsprosjektet.

Plandrevne prosessmodeller følger en utviklingsprosess **steg for steg**, hvor hvert steg må fullføres før man går videre. Dette krever at man har **jobbet godt med planleggingen** på forhånd, da man ikke har mulighet til å gå tilbake til det forrige steget. Eksempelvis når man har samlet alle kravene man trenger til systemet og fortsetter til neste steg i utviklingen, går man ikke tilbake og endrer på kravene. I plandreven utvikling **holder man seg til planen**, helt til ferdig produkt er levert.

Til forskjell fra disse prosessmodellene, er smidige prosessmodeller mer **fleksible**. Her kreves det ikke at man legger en fullstendig plan på forhånd inkludert absolutt alle krav. (**mindre planlegging**). Man kan gjøre ferdig ett steg, **revurdere**, gå tilbake til det forrige steget og endre på det, helt til man er fornøyd.

De fire mest populære smidige praksisene

1. **Daglige Stand-Up møter** – et kort daglig møte som gjennomføres på samme tid, på samme sted hver dag. Hele teamet står og gir en kort «debrief» over hva som har blitt gjort og hvor man er i prosessen.
2. **Retrospektive møter** – hva man har gjort
3. **Sprint planning** – planlegging av et inkrement.
4. **Sprint review** – holdes for å demonstrere et inkrement til produkteier og andre interesserte.

Inkrementell utvikling (smidig)

En vanlig tilnærming i smidige prosessmodeller er inkrementell utvikling. Det vil si at systemet utvikles gradvis i form av nye inkremitter som blir lagt til. Hvert inkrement evalueres før utviklingen av neste inkrement starter. Hvis evalueringen tilsier at noe må endres, går man tilbake og gjør det, kanskje endrer man noen krav, før man går videre til neste inkrement.

Scrum (smidig)

Scrum er en smidig prosessmodell som tar i bruk tidsboksbasert tidsflyt, og er en mye brukt prosessmodell for programvare utvikling i dag. Denne modellen baserer seg på såkalte tidsbokser, som er en fast tidsramme på rundt 2-4 uker. I løpet av denne tidsrammen skal gitte oppgaver fullføres. Tidsboksene kalles også sprint eller iterasjon.

Scrum har et fastsatt start- og avslutningspunkt og består av tre ulike faser:

1. **Planleggingsfasen** – i denne fasen settes mål for prosessen, og oppgaver og arbeidsoppgaver planlegges.

2. **Gjennomføringsfasen** – i denne fasen gjennomfører man selve utviklingen av systemet, hvor disse tidsboksene blir tatt i bruk.
3. **Avslutningsfasen** – i denne fasen dokumenteres prosessen og det blir foretatt av en evaluering oppimot målene som ble satt i planleggingsfasen.

Det finnes både fordeler og ulemper ved å ta i bruk en slik smidig, tidsboksbasert prosessmodell som Scrum.

Fordeler:

1. **Strukturert planlegging** - en av fordelene ved bruk av Scrum er at til tross for at prosessmodellen er smidig, så tar man likevel i bruk strukturert planlegging. Ved å sette et start- og sluttidspunkt vet teamet hva de har å forholde seg til.
2. **Motivasjon og engasjement** - i tillegg til at bruken av tidsbokser/sprintere/iterasjoner kan føre til økt motivasjon og engasjement. Man har alltid mindre mål å jobbe opp mot, noe som gjør at arbeidet fremstilles mer håndterbart.
3. **Alle har god oversikt** - I tillegg til dette legger Scrum opp til god kommunikasjon mellom teamet som gjør at alle i teamet har god oversikt over hva som skjer i prosjektet til enhver tid.

Ulemper:

Til tross for at prosessmodellen er svært strukturert, bringer det også noen ulemper.

1. **Frist** - en ulempe er at man alltid må forholde seg til en **frist**, noe som kan føre til økt press for å fullføre innen den gitte tidsrammen.
2. **Dårligere kvalitet** - i tillegg til dette kan det føre til at kvaliteten på arbeidet ikke blir på sitt beste, da man må jobbe inn mot en satt frist noe som kan være et stressmoment.
3. **Ikke så fleksibel** - samtidig baserer modellen seg på tidsrammer, noe som ikke gjør den like fleksibel som eksempelvis KanBan.

Hva er backlog?

Backlog er en liste av arbeidsoppgaver som skal utføres i prosjektet.

KanBan (smidig)

KanBan er også en smidig prosessmodell, men som baserer seg på å fullføre oppgavene etter behov, som de oppstår på veien. Når en oppgave fullføres, vil behovet for å starte på en ny oppgave oppstå. Skjer det noe midt i en påbegynt oppgave, stopper man opp og løser problemet, for å så fortsette videre. Selve tanken bak KanBan er å få utført oppgavene så fort som mulig og kun produsere ved behov. Denne prosessmodellen er meget flytbasert.

Fordeler:

Det finnes både fordeler og ulemper med en så «fri» smidig prosessmodell.

1. **God flyt** - en av fordelene med denne modellen er at det blir en god flyt i selve prosjektet, da man jobber med oppgaver ettersom behovet oppstår.
2. **Ikke tidspress** - i tillegg er det lite fokus på tidsrammer rundt prosjektet noe som legger til rette for muligheten for grundig arbeid uten preg av stress.
3. **WIP** - KanBan også WIP (Work in progress) som setter en begrensning for pågående arbeid. På denne måten sikrer man at teamet får jobbet effektivt med de oppgavene som er.
4. **Fin modell for uforutsigbare prosjekter** - KanBan er en passende prosessmodell i systemutviklingsprosjekter hvor man ikke har oversikt over alle oppgavene i prosjektet på forhånd.

Ulemper:

1. **Mindre oversikt** - en av ulempene med KanBan er at man ikke vil ha like god oversikt over prosjektet ved oppstart, da det ikke er planlagt på den måte slik at prosjektet har fastsatte tidsrammer og oppgaver.
2. **Tid og ressurskrevende** - i tillegg kan KanBan som prosessmodell være både tid- og ressurskrevende da fokuset ikke ligger på tidsestimering og man har heller ikke kontroll på hvilke oppgaver som vil oppstå underveis i utviklingen.

Hva er forskjellen på Scrum og KanBan?

Både Scrum og KanBan er to smidige prosessmodeller, men likevel er de bygd opp på to svært forskjellige måter.

Til tross for at Scrum er smidig, er den preget av en del planlegging. Man setter et gitt start- og slutt tidspunkt, samtidig som man jobber med såkalt tidsboksbasert. Det vil si at man utfører gitte oppgaver innenfor den gitte tidsrammen. Dette står i kontrast til KanBan hvor man ikke setter et fast start- og sluttidspunkt, og man utfører de ulike oppgavene som oppstår underveis. Tankegangen er at ved en ferdig oppgave, vil behovet for en ny oppgave oppstå.

Kundeinvolvering i utvikling

For å utvikle det mest kompatible systemet er det svært viktig å involvere kundene i utviklingsprosessen. En av fordelene med det er at kunden har god forståelse av domenet og kan komme med gode innspill til utviklingen. Det gjør at systemet blir utviklet slik at det dekker brukerens behov.

To ulemper med kundeinvolvering i utviklingen er at det er svært tidkrevende. I tillegg er det ikke garantert at kunden alltid vil være tilgjengelig når behovet for involveringen oppstår.

Hva er prosess-samsvar?

Prosess-samsvar handler om hvordan man faktisk jobber opp imot hvordan dette samsvarer med modellen.

Hvis man mislykkes i et prosjekt og man ikke vet om man fulgte modellen, så vet man heller ikke om man må endre på modellen. I motsetning til om man følger modellen og det gikk dårlig, så vet man at man kanskje burde endre på modellen.

Hva er branching?

Branching, eller forgrening på norsk, handler om å lage en kopi av kildekoden for å så lage to ulike versjoner av systemet som utvikles separat. Etter at disse grenen har blitt endret av utviklerne, settes de sammen igjen ved en sammenslåing.

DEV-OPS

Hva er DevOps?

DevOps er arbeidsmetode som har som mål å effektivisere arbeidsflyten i virksomheten.

DevOps går ut på at alle er ansvarlige for alt. Alle i temaet jobber med både programvareutviklingen, testingen og utgivelse. I tillegg til dette automatiseres prosessen mest mulig, slik som testing og utgivelse. På denne måten sparer man tid, når dette går automatisk. Samtidig slipper man «kommunikasjons-delay», noe som kan oppstå når flere team arbeider på samme prosjektet. Dermed vil tiden fra ide til utgivelse kortes betraktelig ned ved bruk av DevOps. I tillegg kvalitetssikres systemet, ved blant annet bruk av kontinuerlig integrasjon («continuous integration»). På denne måten blir kodeendringer eller ny kode lagt inn i et «shared respiratory», hvor den blir innlemmet sammen med den allerede eksisterende koden og automatisk testet. Dette gjør at man enkelt kan oppdage feil, slik som integrasjons-feil.

Hva er hovedprinsippene i DevOps?

Hovedprinsippet i DevOps er at alt som kan automatiseres skal automatiseres. På denne måten sparer man både kostander i forhold til integrasjon, testing og utgivelse og man sparer tid på at alt automatiseres.

Et annet fundamentalt prinsipp i DevOps er kontinuerlig integrasjon. Dette prinsippet går ut på at kode deles i et «shared respiratory» flere ganger om dagen, hvor hver «check in» blir automatisk testet noe som gjør at man slipper eventuelle integrasjonsproblemer med den allerede eksisterende koden.

Kontinuerlig integrasjon, levering og utgivelse.

Kontinuerlig integrasjon, kontinuerlig levering og kontinuerlig utgivelse er tre konsepter som brukes innenfor programvareutvikling, hvor målet er å forbedre både hastighet og kvalitet.

Kontinuerlig integrasjon er en praksis hvor man deler koden i et «shared respiratory» flere ganger daglig. For hver «check in» blir koden automatisk testet, noe som fanger opp integreringsproblemer ved koden. Dette gjør at alle delene av kode blir samlet til en kjørbare versjon av systemet. På denne måten vil det bli enklere å løse eventuelle feil, da koden hele tiden testes.

Kontinuerlig levering kan ansees som en slags utvidelse av kontinuerlig integrasjon. Her koder man sekvens for sekvens hvor koden gjennomgår automatisk tester. Forskjellen er at kodeendringene automatisk distribueres til et produksjonsmiljø, hvor produktets driftsmiljø simuleres og den kjørbare programvaren testes. Selv om programvaren består testingen i produksjonsmiljøet, er det ikke garantert at denne versjonen blir gitt ut. Bruk av continuous delivery gjør at man kan oppdage feil som kanskje ikke oppdages i testmiljøet, i tillegg til at man sikrer seg at det nye systemet er klar for utgivelse til brukerne.

Kontinuerlig utgivelse blir automatisering tatt enda mer i bruk. Her blir kodeendringene automatisk testet og automatisk sluppet til kunden. Det eneste som kan forhindre utgivelse av programvarene er om den ikke består de automatiserte testene. I tillegg fører kontinuerlig utgivelse til at nye kodeendringer fører til nye tester og automatisert slipp av nye kodeendringer.

Hvilke prinsipper fra smidig utvikling er tett koblet opp som DevOps som praksis?

Dev-Ops og smidig utvikling til programvareutvikling som deler en del prinsipper.

1. **Alle jobber med alt** – smidig utvikling baserer seg på tverrfaglige team, noe også DevOps gjør. Dermed vet alle alt og man blir spare tid.
2. **Inkrementell utvikling** – smidig utvikling fremmer en iterativ tilnærming av programvareutviklingen hvor man jobber inkrement til inkrement. Dette bygger DevOps på, noe som gjør at man får gjennomgått hyppige testinger og utrullinger av programvare i produksjonsmiljøet.
3. **Kontinuerlig levering** – smidig utvikling bruker hyppig levering av fungerende programvare, noe DevOps også tar i bruk samtidig som de implementerer automatisk testing og implementasjon.

KRAV OG KRAVHÅNDTERING I SYSTEMUTVIKLING

Kravhåndteringsprosessen/kravspesifikasjon

Ved utvikling av et system er det svært viktig å fastsette kravene til det planlagte systemet. Når vi utvikler et IT-system løser vi et problem eller utnytte mulighetene og kravene forteller oss nettopp hva som skal lages.

Dette kalles for kravhåndtering, og inkluderer å identifisere, analysere og spesifisere kravene til det nye eller forbedrede systemet. Vi skiller mellom funksjonelle og ikke-funksjonelle krav.

Hva er et brukerkrav?

Forskjellige typer krav er nødvendig for å kommunisere informasjon om systemet til forskjellige lesere. Man trenger å skrive kravene på forskjellige nivåer av hvor detaljert de er fordi forskjellige typer av lesere eller mottakere skal forstå. Derfor deler vi inn i brukerkrav og system krav.

Et brukerkrav er et krav skrevet i et naturlig, gjenkjennelig språk for de aller fleste. De som har behov for brukerkravene, har ikke behov for å vite hvordan systemet skal bli implementert, de trenger kun å vite hvilke funksjoner systemet skal tilby en bruker.

Hva er et systemkrav?

Et systemkrav er et krav som definerer nøyaktig hva som skal implementeres.

Funksjonelle krav

Funksjonelle krav beskriver hva systemet skal gjøre eller ikke gjøre. Det kan være hvilke tjenester systemet skal tilby eller hvordan systemet skal reagere på ulik input. Hvilke mål skal det tilfredsstille?

Eksempler:

- Systemet skal ha funksjon for forhåndskjøp av billetter.
- Systemet skal ha en mulighet for å legge til ønskede kinovisninger for en ansatt.
- Systemet skal ha en funksjon som viser salkart til ønsket kinovisning, som gir en oversikt over ledige seter i salen.

Ikke-funksjonelle krav

Ikke-funksjonelle krav kan deles opp i tre forskjellige typer: produktkrav, organisatoriske krav og eksterne krav.

Produktkrav handler om hvordan produktet skal fungere.

1. Brukervennlighet – det skal være enkelt for alle, uansett funksjonsnedsettelse, å navigere enkelt på nettsiden. Produktet skal utvikles etter disse retningslinjene, det skal utvikles i henhold til universell utforming, slik at alle kan dra nytte av systemet uansett funksjonsnedsettelse.
2. Sikkerhet – systemet skal ha god sikkerhet. Brukeren skal ikke være i tvil.

Organisatoriske krav handler om hvordan systemutviklingsprosessen skal gjennomføres.

1. Koden skal være godt dokumentert slik at det skal være enkelt for utviklere å vedlikeholde/oppdatere systemet i framtiden.
2. Leveransetidspunkt.
3. Kostnader og ressurser.

Eksterne krav handler om hvilke krav omgivelsene har.

1. Systemet/nettsiden skal oppfylle WCAG 2.1 kravene slik at nettsiden er universelt utformet. Dette gjør at nettsiden er brukervennlig, alle skal kunne ta i bruk nettsiden til tross for funksjonsnedsettelse.

KRAV OM INFORMASJONSSIKKERHET & RISIKO

Hva er informasjonssikkerhet?

Informasjonssikkerhet dreier seg om å håndtere risiko relatert til virksomhetens informasjonsverdier og behandling av personopplysninger. Informasjonen skal være sikret.

Hvorfor er det et økende behov for informasjonssikkerhet?

I dagens samfunn er det et økende behov for informasjonssikkerhet på grunn av økende digitalisering. Teknologien blir tatt mer og mer i bruk, og derav flere sikkerhetstrusler også.

KIT

Informasjonssikkerhet kan deles inn i tre prinsipper. Det handler om konfidensialitet – at informasjon ikke blir kjent for uvedkommende, integritet – at informasjon ikke blir endret utilsiktet eller av uvedkommende og tilgjengelighet – at informasjon er tilgjengelig ved behov.

Risiko ved utviklingsprosjekt

Med utviklingsprosjekt kommer følger det alltid risikoer. Risikoene kan deles inn i tre forskjellige kategorier: prosjektrisiko, produktrisiko og forretningsrisiko.

Prosjektrisiko handler om tidsplan/ressurser

Produktrisiko handler om kvaliteten og programvaren som utvikles

Forretningsrisiko handler om organisasjonen som utvikler/eier programvare

Risikoanalyse ved usikkerhetsmatrise(risikomatrise)

En usikkerhetsmatrise viser frem ulike risikoer ved et utviklingsprosjekt. Det er viktig å få med de tre hovedtypene av risiko: prosjekt-, produkt- og forretningsrisiko.

Tabellen skal bestå av de fire kolonnene risiko, sannsynlighet, konsekvens og tiltak.

Eksempel:

Risiko	Sannsynlighet	Konsekvens	Tiltak
Sykdom	Svært høy	Alvorlig	Tilgang på vilkarbyrå
Budsjettproblemer	Lav	Alvorlig	Endre på prosjektet
Programvarefeil	Moderat	Mindre alvorlig	Grundig testing
Personopplysninger på avveie	Lav	Katastrofal	Grundig testing
Konflikt innad i teamet	Høy	Alvorlig	Teambulding

Hvilke fundamentale aktiviteter utføres i systemutvikling utover programmering?

Bortsett fra programmering, så består systemutvikling av:

- Planlegging
- Kravarbeid i form av kravinnsamling, kravanalyse og kravspesifikasjon
- Utforming og design
- Testing
- Dokumentering
- Innføring, vedlikehold og videreutvikling

UML-MODELLERING

OBS! Det er svært viktig at alle diagrammene for et og samme system samsvarer med hverandre. En metode som blir brukt i et sekvensdiagram MÅ være med i et klassediagram. Lag derfor klassediagram parallelt med sekvensdiagrammer.

Hva er UML-modellering?

UML-modellering er et objektorientert modelleringsspråk, som brukes i analyse og design av datasystemer. En viktig del av programvareutviklingen.

Hva er generalisering i UML? Eksempler.

Generalisering i UML er en måte å håndtere kompleksitet. Hvis to klasser har felles egenskaper, settes den ene klassen som «hoved-klasse» og den andre klassen arver egenskapene fra hovedklassen, som en subklasse. Eksempelvis klassen person blir superklassen, og klassen student blir en subklasse og arver på klassen person. I et klassediagram vises dette med en pil fra subklassen til superklassen.

Tekstlig beskrivelse

Man blir ofte bedt om å gi en tekstlig beskrivelse for et brukstilfelle/UseCase, før man viderefører dette til et sekvensdiagram. Et slikt brukstilfelle kan være «Opprett ny bruker i systemet». En tekstlig beskrivelse tar for seg interaksjonen mellom bruker og systemet, ved en nummerert liste som beskriver hvert interaksjonssteg.

Dette må med: navn, primæraktør, sekundæraktør, pre- og postbetingelser, hoved- og alternativ flyt. Postbetingelsen er «målet» med interaksjonen.

Eksempel:

Navn: Registrere ny bruker via nettside

Primæraktør: Bruker

Sekundæraktør: Ingen

Prebetingelser: Ingen

Postbetingelser: Ny bruker opprettet i systemet

Hovedflyt:

1. Bruker velger "registrer ny bruker"
2. Systemet ber bruker om å oppgi navn, tlfnr, alder og mailadresse
3. Bruker skriver inn informasjon
4. Systemet validerer informasjonen
5. Systemet ber bruker om å opprette passord
6. Bruker skriver inn valgt passord
7. Systemet validerer passord
8. Systemet legger inn bruker i systemet
9. Systemet sender bekreftelse på skjerm og mail

Alternativ flyt:

- 4.1. Ugyldig tlfnr, for få siffer
- 4.2. Returnerer tilbake til steg 2
- 7.1. Ugyldig passord, mangler vanskelighetsgrad
- 7.2. Systemet ber bruker taste inn et annet passord
- 7.3. Returnerer til steg 6

Sekvensdiagram

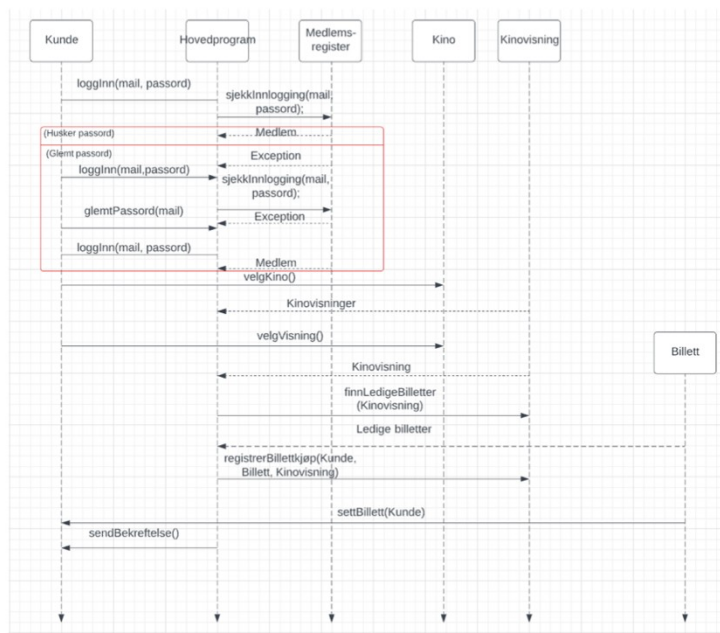
Et sekvensdiagram viser interaksjon og informasjonsflyt mellom aktører, systemet og systemkomponentene i form av objektklasser.

Et sekvensdiagram brukes når man ønsker å se interaksjonen på et mer kodenært nivå.

Dette må med:

1. Alle klassene som firkanter
2. Pil med metodekall
3. Pil med returverdier i stiplelinje.
4. Alternativ flyt i rød boks, som viser de to tilfellene.

Eksempel:



Aktivitetsdiagram

Et aktivitetsdiagram viser hendelsesflyten i en use-case grafisk. Diagrammet viser hvordan flyten i prosessen foregår, hvor stegene vises som aktiviteter representert av rektangler og beslutninger underveis som diamanter.

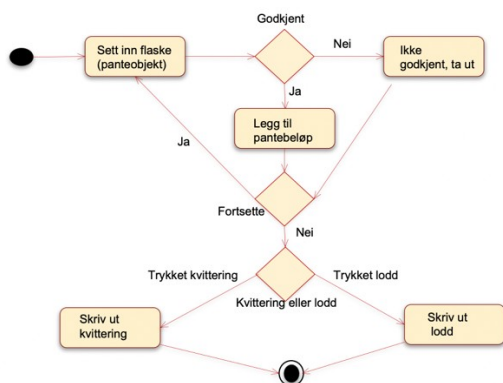
Et sekvens- og aktivitetsdiagram er veldig likt, forskjellen ligger i at sekvensdiagrammet er mer rettet mot selve koden, i motsetning til aktivitetsdiagrammet som er mer forretningsrettet. Det visualisere systemet funksjonalitet og er nyttig for at utviklere skal kommunisere med kunder. En kunde har mye mer nytte av et aktivitetsdiagram som er et høyere abstraksjonsnivå enn et sekvensdiagram som er mer kodenært.

Dette må med:

1. Start og slutt markeres med to rundinger.
2. Aktivitet skrives som en firkant med tekst.
3. Valg som en diamant, med piler ut fra diamanten med alle valgmulighetene.

Eksempel:

Aktivitetsdiagram – pante flasker



Klassediagram

Et klassediagram viser strukturen i systemet. Objektklasser av et system, med deres attributter (instansvariabler) og metoder. En domenemodell fungerer som et forenklet klassediagram, hvor metoder ikke er inkludert.

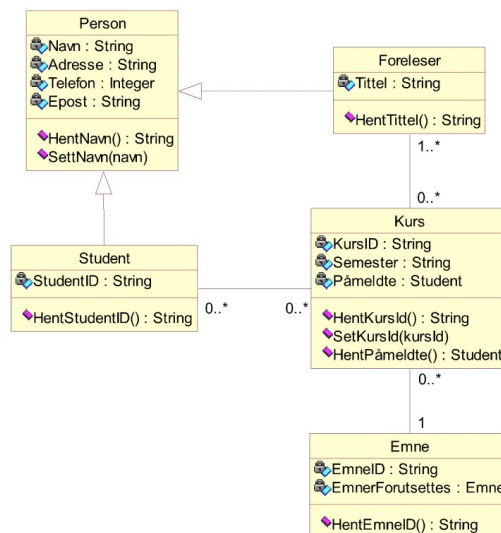
Klassediagrammet brukes når man ønsker å få en oversikt over arkitekturen i systemet, alle klasser, metoder og assosiasjoner i systemet.

Dette må med:

1. Klasser med navn, navn på instansvariabel og type.
2. Metoder med returverdi og parametere.
3. Subklasser representeres med pil fra til.
4. Andre «connections» mellom klasser med en strek mellom klassene.
5. Multiplisitet

Eksempel:

Klassediagram - Kurs inngår i Emne



Hva er assosiasjon og multiplisitet i UML?

I klassediagram bruker vi både assosiasjon og multiplisitet.

Assosiasjoner handler om hvordan de ulike klassene er relatert til hverandre, og vises ved en strek fra en klasse til en annen.

Multiplisitet angis i enden av assosiasjonen og angir hvor mange koblinger som kan/må gå fra et objekt fra den ene klassen til et objekt i den andre klassen.

Eksempelvis klassediagrammet over. En kvittering må ha et pantesystem å komme fra, men et pantesystem MÅ ikke ha kvitteringer.

Notasjon:

0..* - Ingen eller flere

1 – Tilhører ett objekt.

Kohesjon

NB! Lav og høy, tenk motsatt av normalen.

Kohesjon er et mål på hva slags ansvar et objekt har og hvor fokusert det ansvaret er.

Klassediagram og domenemodeller hjelper oss å finne grad av kohesjon, og kobling.

Høy kohesjon – et objekt med høy kohesjon har moderat ansvar og utfører et begrenset antall oppgaver innenfor et funksjonelt område.

Lav kohesjon – et objekt med lav kohesjon har ansvar for mange oppgaver innen flere ulike funksjonelle områder

Kobling

Kobling er et mål på hvor sterkt et objekt er knyttet til andre objekter.

Høy kobling – et objekt med høy kobling er avhengig av mange andre objekter, noe som kan gjøre kodeendring ekstra vanskelig og komplisert.

Lav kobling – et objekt med som ikke er «koblet» til så mange andre objekter. Enklere å gjøre endringer rundt denne klassen.

MVC – Model View Controller

MVC er en type «programmeringsteknikk» hvor man deler opp koden i tre forskjellige deler, det er en type designpattern.

Model – holder på dataen, for eksempel et personobjekt som er en instans av en klasse med definerte egenskaper og metoder.

View – viser fram dataen.

Controller – flytter på data mellom view og model. For eksempel ved et muse/taste-trykk.

Frontend-Backend

Front- og backend er et annet type designpattern, hvor koden deles inn i to forskjellige deler, front- og backend. Frontend står for brukerinterfacet og alt brukeren ser, typisk HTML og CSS i webapplikasjoner. Backend er selve dataen, eller «serversiden» av koden, og er ofte kodet ved hjelp av Java eller Python.

Hva er forskjellen på MVC og Frontend-Backend?

Både MVC og front/backend deler opp og organiserer koden i ulike deler, men på forskjellige måter. MVC er spesifisert mot brukerinterfacer, i motsetning til Frontend-Backend som er en bredere plan som tar for seg hele applikasjonen. MVC implementeres sammen med en Frontend-del, men i Frontend-Backend deler hele applikasjonen inn i en del Frontend, en del Backend.

IT-KONTRAKTER

Hva er en kontrakt?

En kontrakt er en avtale som mellom partene etablerer en bindende forpliktelse til å gjøre eller unnlate å gjøre noe.

Hvilke utfordringer støter man på ved IT-kontraktene?

1. Man snakker om abstrakte og usynlige systemer, spesielt for brukeren. Dette kan gjøre det vanskelig å kommunisere godt med kunden.
2. Kunden vet ikke alltid hva den ønsker, fordi den ikke har god nok kunnskap om temaet.
3. Kompleksitet